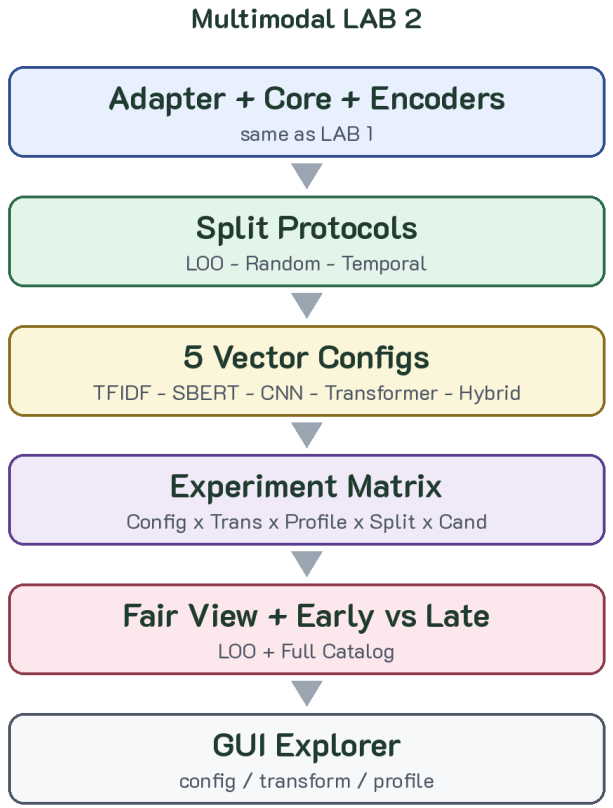
LAB 5:
Multimodal Experiments

การทดลองเชิงเปรียบเทียบ

204729
Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

Multimodal Experimental Comparison & Evaluation



อธิบายแนวคิดของ Source Code ทีละ Part

Encoder x Transformation x Profile x Split x Candidate x Fusion

COMPUTER SCIENCE
Chiang Mai University

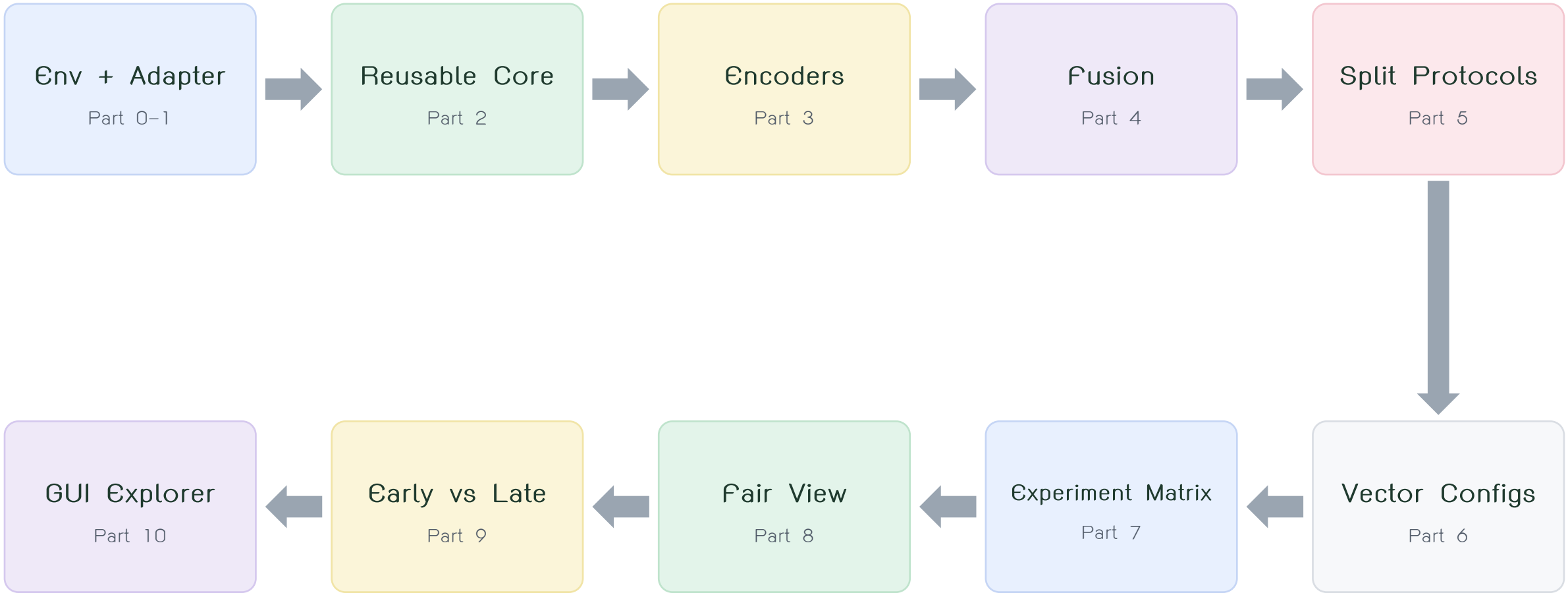
Multimodal LAB 2 ขยายจาก LAB 1 อย่างไร

ภาพรวม

- **Part 0–4 เหมือน LAB 1 ทุกบรรทัด** (Adapter, Reusable Core, 9 Encoders + `VECTOR_LIBRARY`, `early_fusion` / `late_fusion_rank` / `evaluate_late_fusion`) — เล่มนี้ไม่เพิ่ม encoder หรือ metric
- **สิ่งใหม่ 1** — `SPLITS` (Part 5): LOO, Random Holdout, Temporal (ถ้ามี timestamp)
- **สิ่งใหม่ 2** — `CONFIGS` (Part 6): 5 ชุดเวกเตอร์ = Text-TFIDF, Text-SBERT, **CNN-Multimodal** (TextCNN+ResNet+LogMel-CNN+Video-CNN), **Transformer-Multimodal** (SBERT+ViT+Wav2Vec2+Video-ViT), **Hybrid** (`X_fused` ของ LAB 1)
- **สิ่งใหม่ 3** — **Experiment Matrix** (Part 7): Config × Transform × Profile × Split × Candidates = 5 × 2 × 3 × 2 × 2 = 120 configuration ผ่าน `evaluate_vector_space` เดิม
- **Part 8 Fair View** กรอง LOO + Full Catalog ก่อนเทียบ ; **Part 9 Early vs Late** ต่อ profile ด้วย encoder ชุดเดียวกัน ; **Part 10 GUI Explorer** เลือก config / transformation (รวม PCA+L2) / profile

คำถามของเล่มนี้: “ครอบครัว encoder (CNN vs Transformer vs Hybrid) เทียบกับ Text-only ภายใต้เงื่อนไขควบคุมเป็นอย่างไร”

โครงสร้าง Notebook Multimodal LAB 2



Part 0–4 = โค้ดเดิม ; Part 5–10 = การออกแบบการทดลอง

แผนที่ของ Source Code Multimodal LAB 2

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	ต่างจาก LAB 1
0–1	Environment + Dataset Adapter	`load_dataset`, `items`, `ratings`, `ITEM_INDEX`	เหมือนเดิม
2	Reusable Core	`transform_vectors`, `make_profile`, `rank_by_vector`, metrics, `split_*`, `evaluate_vector_space`, `quick_eval`	เหมือนเดิม (ดูสไลด์ LAB 1)
3	9 Encoders	`VECTOR_LIBRARY`	เหมือนเดิม
4	Fusion	`early_fusion`, `DEFAULT_MODALITIES`, `X_fused`, `late_fusion_rank`, `evaluate_late_fusion`	เหมือนเดิม
5	Split protocols	`SPLITS = {LOO, Random[, Temporal]}`	ใหม่: ใช้ `split_*` จาก Core
6	Vector configurations	`CONFIGS` (5 ชุด)	ใหม่: CNN-family / Transformer-family / Hybrid
7	Experiment matrix	`RUN_*`, `experiment_results`	ใหม่: loop 5 ชั้น
8	Fair comparison view	`view` (LOO + Full Catalog) sort by nDCG@3	ใหม่
9	Early vs Late fusion	`evaluate_vector_space(X_fused)` vs `evaluate_late_fusion(DEFAULT_MODALITIES)` ต่อ profile	ใหม่
10	GUI Explorer	`gui_exp`, `gr.Interface`	เลือก config + 4 transformation

1

ส่วนที่ใช้ซ้ำจาก LAB 1

Part 0 – Part 4: ทบทวนเฉพาะ interface ที่ LAB 2 เรียกใช้

- `transform_vectors(X, mode)` — Raw / L2 / Center+L2 / Standard+L2 / PCA+L2
- `evaluate_vector_space(X, label, profile_method, k, folds, candidate_mode, n_neg)` ⟶ `(summary, detail)`
- `VECTOR_LIBRARY` ๙ เวกเตอร์ ; `early_fusion(dict)` ; `evaluate_late_fusion(dict, label, profile_method)`

ทบทวน interface ของ Core, Encoders และ Fusion (โค้ดเต็มใน LAB 1) (1/2)

```
# Part 2 — Reusable Core
def transform_vectors(X, mode="L2", pca_dim=16): ...        # Raw | L2
        | Center+L2 | Standard+L2 | PCA+L2
PROFILE_METHODS = ["Positive-only", "Rating-centered", "Rocchio"]
def make_profile(uid, X, rdf, method="Rating-centered", alpha=1.0,
        beta=1.0): ...
def rank_by_vector(uid, X, rdf, method="Rating-centered",
        candidates=None, alpha=1.0, beta=1.0):
    ...; return ranked, s, p
def split_loo(rdf) / split_random_holdout(rdf, seed=42) /
        split_temporal(rdf)   # -> [(uid, train, heldout)]
def evaluate_vector_space(X, label="",
        profile_method="Rating-centered", k=3, folds=None,
                        candidate_mode="Full Catalog", n_neg=20,
                                alpha=1.0, beta=1.0, seed=42):
    ...; return pd.Series(summary_with_Label), detail

# Part 3 — Encoders (cache .npy)
VECTOR_LIBRARY = {"Text-TFIDF": X_text_tfidf, "Text-CNN": X_text_cnn,
```

อธิบายโค้ด

จุดที่ LAB 2 ใช้ประโยชน์

- `evaluate_vector_space` รับ `folds` และ `candidate_mode` จากภายนอก ⟶ Part 7 ส่ง `SPLITS[sp]` และ `cand` เข้าไปโดยไม่แก้ engine
- รับ `X` ใด ๆ ⟶ Part 6 ประกอบ `CONFIGS` จาก `VECTOR_LIBRARY` ด้วย `early_fusion` แล้วส่งเข้าไป
- `label` ช่วยระบุแถวเมื่อรวมตาราง ; Part 7 ใช้ `f"{cfg}/{trans}"`
- `evaluate_late_fusion` มี signature ขนานกับ `evaluate_vector_space` ⟶ Part 9 เทียบ Early/Late ต่อ profile ได้ตรง ๆ
- ทุกอย่างถูกโหลดจาก cache `.npy` ⟶ เล่มนี้รันเร็วแม้มี 120 configuration

ทบทวน interface ของ Core, Encoders และ Fusion (โค้ดเต็มใน LAB 1) (2/2)

```
        "Text-SBERT": X_text_sbert,
                  "Image-CNN": X_image_cnn, "Image-ViT": X_image_vit,
                  "Audio-CNN": X_audio_cnn, "Audio-Wav2Vec2":
                         X_audio_transformer,
                  "Video-CNN": X_video_cnn, "Video-ViT": X_video_vit}

# Part 4 — Fusion
def early_fusion(vector_dict, weights=None): ...            # L2 blocks
        * sqrt(w) -> hstack -> L2
DEFAULT_MODALITIES = {"text": X_text_sbert, "image": X_image_cnn,
        "audio": X_audio_transformer, "video": X_video_vit}
X_fused = early_fusion(DEFAULT_MODALITIES)
def late_fusion_rank(uid, vector_dict, rdf, profile_method,
        weights=None, candidates=None, alpha=1, beta=1): ...
def evaluate_late_fusion(vector_dict, label, profile_method, k=3,
        folds=None, weights=None,
                        candidate_mode="Full Catalog", n_neg=20,
                                seed=42): ...
```

อธิบายโค้ด

จุดที่ LAB 2 ใช้ประโยชน์

- `evaluate_vector_space` รับ `folds` และ `candidate_mode` จากภายนอก ⟶ Part 7 ส่ง `SPLITS[sp]` และ `cand` เข้าไปโดยไม่แก้ engine
- รับ `X` ใด ๆ ⟶ Part 6 ประกอบ `CONFIGS` จาก `VECTOR_LIBRARY` ด้วย `early_fusion` แล้วส่งเข้าไป
- `label` ช่วยระบุแถวเมื่อรวมตาราง ; Part 7 ใช้ `f"{cfg}/{trans}"`
- `evaluate_late_fusion` มี signature ขนานกับ `evaluate_vector_space` ⟶ Part 9 เทียบ Early/Late ต่อ profile ได้ตรง ๆ
- ทุกอย่างถูกโหลดจาก cache `.npy` ⟶ เล่มนี้รันเร็วแม้มี 120 configuration

2

Split Protocols และ Vector Configurations

Part 5 – Part 6: ตัวแปรต้นสองมิติใหม่

- `SPLITS` จาก `split_loo` / `split_random_holdout` / `split_temporal`
- `CONFIGS` 5 ชุด: Text-only 2 ชุด + multimodal 3 ครอบครัว

Part 5: SPLITS

```
SPLITS = {"LOO": split_loo(ratings), "Random":
        split_random_holdout(ratings)}
T = split_temporal(ratings)
if T is not None: SPLITS["Temporal"] = T
print({k: len(v) for k, v in SPLITS.items()})
```

อธิบายโค้ด

สามวิธีเลือก held-out (ฟังก์ชันอยู่ใน Core)

- LOO: positive ที่ rating สูงสุด ; Random: positive สุ่ม (seed 42) ; Temporal: positive ล่าสุด — ต้องมีคอลัมน์ `timestamp` ไม่งั้นคืน `None` และไม่ถูกใส่ใน dict
- ทุก protocol คืน `[(uid, train, heldout)]` รูปแบบเดียวกัน ⟶ ส่งเป็น `folds=` ได้ทันที
- print จำนวน fold ⟶ LOO และ Random มีจำนวนเท่ากัน (ผู้ใช้ที่มี positive) แต่ held-out อาจต่างกัน
- **Candidate protocol** (Full Catalog / Sampled Negatives) เป็นมิติที่ 5 ซึ่งอยู่ใน `evaluate_vector_space` อยู่แล้ว

Part 6: CONFIGS — 5 ชุดเวกเตอร์

```
CONFIGS = {
    "Text-TFIDF": transform_vectors(X_text_tfidf, "L2"),
    "Text-SBERT": X_text_sbert,
    "CNN-Multimodal": early_fusion({
        "text": X_text_cnn, "image": X_image_cnn, "audio": X_audio_cnn,
                "video": X_video_cnn}),
    "Transformer-Multimodal": early_fusion({
        "text": X_text_sbert, "image": X_image_vit, "audio":
                X_audio_transformer, "video": X_video_vit}),
    "Hybrid-Multimodal": X_fused,      # SBERT + ResNet + Wav2Vec2 +
            ViT (DEFAULT_MODALITIES)
}
print({k: v.shape for k, v in CONFIGS.items()})
```

อธิบายโค้ด

ออกแบบชุดทดลองให้ตอบคำถามเดียวชัด ๆ

- **Text-only baseline 2 ชุด**: TF-IDF (lexical) และ SBERT (Transformer) — ใช้เทียบว่า “เพิ่มภาพ/เสียง/วิดีโอแล้วดีขึ้นไหม”
- **CNN-Multimodal**: ทุกสื่อผ่านครอบครัว CNN (TextCNN, ResNet18, Log-Mel⟶ResNet18, frames⟶ResNet18) มิติ 96+512+512+512
- **Transformer-Multimodal**: ทุกสื่อผ่าน Transformer (SBERT, ViT, Wav2Vec2, frames⟶ViT) มิติ 384+768+768+768
- **Hybrid**: เลือกตัวที่ “น่าจะดีที่สุด” ต่อสื่อ (SBERT, ResNet, Wav2Vec2, ViT) = `X_fused` ของ LAB 1
- ทุกชุดผ่าน `early_fusion` (√w = 1 ทุกสื่อ) และ L2 ⟶ เข้า `transform_vectors` ใน Part 7 ต่อได้

5 configuration ในมุมของการทดลอง

Config	Text	Image	Audio	Video	คำถามที่ตอบ
Text-TFIDF	TF-IDF	—	—	—	baseline บทที่ 3
Text-SBERT	SBERT	—	—	—	Transformer ข้อความอย่างเดียวพอไหม
CNN-Multimodal	TextCNN	ResNet18	LogMel → ResNet18	frames → ResNet18	ครอบครัว CNN ทั้งระบบ
Transformer-Multimodal	SBERT	ViT-B/16	Wav2Vec2	frames → ViT	ครอบครัว Transformer ทั้งระบบ
Hybrid-Multimodal	SBERT	ResNet18	Wav2Vec2	ViT	ผสมตัวที่เหมาะกับแต่ละสื่อ

Text-CNN อ่อนมาก (ฝึกจาก 12 เรื่อง) → CNN-Multimodal มักเสียเปรียบตั้งแต่ต้น — ต้องตีความอย่างเป็นธรรม

3

Experiment Matrix และ Fair View

Part 7 – Part 8: loop 5 ชั้น แล้วกรองก่อนเปรียบเทียบ

- `experiment_results` = 120 แถว (150 ถ้ามี Temporal)
- `view` = LOO + Full Catalog เรียงตาม nDCG@3

Part 7: RUN_* และ loop 5 ชั้น

```
RUN_CONFIGS = ["Text-TFIDF", "Text-SBERT", "CNN-Multimodal",
        "Transformer-Multimodal", "Hybrid-Multimodal"]
RUN_TRANS = ["L2", "Center+L2"]
RUN_PROFILES = PROFILE_METHODS
RUN_SPLITS = list(SPLITS)
RUN_CANDIDATES = ["Full Catalog", "Sampled Negatives"]

rows = []
for cfg in RUN_CONFIGS:
    for trans in RUN_TRANS:
        X = transform_vectors(CONFIGS[cfg], trans)
        for prof in RUN_PROFILES:
            for sp in RUN_SPLITS:
                for cand in RUN_CANDIDATES:
                    s, _ = evaluate_vector_space(X, f"{cfg}/{trans}",
                            prof, k=3, folds=SPLITS[sp],
                                        candidate_mode=cand,
                                                n_neg=20)
                    rows.append({"Config": cfg, "Transform": trans,
                            "Profile": prof,
                                 "Split": sp, "Candidates": cand,
                                        **s.to_dict()})
experiment_results = pd.DataFrame(rows)
display(experiment_results.round(4))
```

อธิบายโค้ด

หนึ่งแถว = หนึ่ง experimental condition

- 5 × 2 × 3 × 2 × 2 = 120 แถว ; `transform_vectors` คำนวณครั้งเดียวต่อ (cfg, trans) แล้วใช้ซ้ำในชั้นใน
- `k=3`, `n_neg=20`, `seed=42` คงที่ ⟶ ผลต่างมาจาก 5 มิติที่ตั้งใจ ; แต่ละแถวมี 5 คอลัมน์เงื่อนไข + 13 metric + `Label`
- `RUN_*` แก้ได้: เพิ่ม `"Standard+L2"`, `"PCA+L2"` หรือ config ใหม่ (เช่น text+image เท่านั้น) แล้วรันซ้ำ — ค่าตั้งต้นตั้งใจให้รันได้ในคาบเรียน
- **Sampled Negatives กับ 12 เรื่อง**: negative มีไม่ถึง 20 ⟶ candidate ≈ Full ; มีไว้เพื่อให้เห็นโครงของ protocol
- ตารางเป็น long format ⟶ กรองด้วย pandas ทีละมิติ

Part 8: Fair Comparison View

```
view = experiment_results[
    (experiment_results["Candidates"] == "Full Catalog") &
    (experiment_results["Split"] == "LOO")
]
cols = ["Config", "Transform", "Profile", "HR@3", "MRR", "nDCG@3",
        "AUC",
        "Coverage", "Gini", "ILD", "Novelty", "Calibration"]
display(view[[c for c in cols if c in
        view.columns]].sort_values("nDCG@3", ascending=False).round(4))
```

อธิบายโค้ด

กรองก่อน เทียบทีหลัง

- ตรึง protocol ที่ **LOO + Full Catalog** (ยากและสมจริงที่สุด) ⟶ เหลือ 30 แถว = Config × Transform × Profile
- เรียงตาม nDCG@3 ⟶ เห็นว่า config/transformation/profile ใดจัดอันดับดีที่สุดภายใต้เงื่อนไขเดียวกัน
- อ่านคู่กับ Coverage/Gini/ILD/Novelty/Calibration: multimodal อาจ nDCG ใกล้ SBERT แต่ Coverage/ILD ต่างกัน
- **ILD ระวัง**: วัดใน space ของ config นั้น (มิติ 2,400+ ของ fused vs 384 ของ SBERT) จึงเทียบข้าม config ไม่ตรง
- คำถามชี้นำ: Hybrid ชนะ Transformer-only ไหม? Center+L2 ช่วย fused space ไหม? Rocchio ต่างจาก Positive-only เมื่อมี negative rating หรือไม่?

การอ่าน experiment_results ทีละมิติ

Config (encoder family)

- Text-SBERT vs Hybrid: เพิ่มสื่อแล้ว nDCG ดีขึ้นหรือแค่ Coverage เปลี่ยน?
- CNN vs Transformer family: ต่างเพราะ encoder หรือเพราะ TextCNN อ่อน?
- ลอง config ที่ไม่มี text

Transform × Profile

- Center+L2 ช่วย embedding จาก Transformer (anisotropy) แต่กับ TF-IDF อาจไม่
- Rating-centered / Rocchio ใช้ negative ⟶ Coverage/Gini ต่างจาก Positive-only?
- α, β คงที่ = 1 ใน grid

Split × Candidates

- LOO vs Random: held-out ต่างกัน ⟶ ตัวเลขต่างเพราะโจทย์
- Sampled vs Full: ต่างน้อยเพราะ catalog เล็ก — สังเกต AUC
- รายงานทุกครั้งระบุ Split + Candidates + K

ตารางเดียวตอบได้หลายคำถาม ถ้าเปลี่ยนทีละตัวแปร

4

Early vs Late Fusion และ GUI Explorer

Part 9 – Part 10

- encoder ชุดเดียวกัน (`DEFAULT_MODALITIES`) ⟶ ต่างกันเฉพาะ fusion strategy
- GUI เลือก Vector Configuration / Transformation (4 แบบ) / Profile / K

Part 9: Early vs Late Fusion ต่อ profile

```
rows = []
for prof in PROFILE_METHODS:
    se, _ = evaluate_vector_space(X_fused, "Early", prof)
    sl, _ = evaluate_late_fusion(DEFAULT_MODALITIES, "Late", prof)
    rows.append({"Fusion": "Early", "Profile": prof, **se.to_dict()})
    rows.append({"Fusion": "Late", "Profile": prof, **sl.to_dict()})
display(pd.DataFrame(rows).round(4))
```

อธิบายโค้ด

ควบคุม encoder ให้เหมือนกัน

- `X_fused = early_fusion(DEFAULT_MODALITIES)` และ `evaluate_late_fusion(DEFAULT_MODALITIES)` ใช้ 4 **เวกเตอร์ชุดเดียวกัน** ⟶ ผลต่างมาจากจุดที่รวม (ก่อน/หลัง cosine) และวิธีสร้างโปรไฟล์ (1 โปรไฟล์ใน space รวม vs 4 โปรไฟล์แยกสื่อ)
- protocol เดียวกัน (LOO, Full Catalog, K=3) ; วนทั้ง 3 profile ⟶ 6 แถว
- Late fusion น้ำหนัก 1/4 เท่ากัน = ค่าเฉลี่ยของ cosine 4 ค่า ; Early ด้วย √1 = ต่อเวกเตอร์เท่ากัน — ทางคณิตศาสตร์ dot product ของ early = $\sum$ cos_m แต่โปรไฟล์ถูก L2 ใน space รวม จึงไม่เท่ากันเป๊ะ
- ILD ของ Late วัดใน early space (`Xdiv`) ⟶ เทียบกับ Early ได้ตรง
- ต่อยอด: ใส่ `weights={"text": 0.5, ...}` ให้ทั้งสองฟังก์ชันเพื่อทดลองน้ำหนักสื่อ

Part 10: gui_exp — Multimodal Experiment Explorer (1/2)

```
import gradio as gr

def gui_exp(uid, cfg, trans, profile, k):
    X = transform_vectors(CONFIGS[cfg], trans)
    ranked, s, p = rank_by_vector(uid, X, ratings, profile)
    top = ranked[:int(k)]
    hist = ratings[ratings["user_id"].astype(str) == str(uid)].merge(
        items[["item_id", "title", "genre"]], on="item_id",
                how="left")[["title", "genre", "rating"]]
    rec = []
    for j, iid in enumerate(top, 1):
        r = items.iloc[ITEM_INDEX[iid]]
        rec.append({"Rank": j, "Title": r["title"], "Genre":
                r["genre"], "Score": round(float(s[ITEM_INDEX[iid]]),
                        4)})
    return hist, pd.DataFrame({"dim": range(min(20, len(p))), "value":
```

อธิบายโค้ด

สำรวจ configuration ด้วยตา

- เลือก Config จาก `CONFIGS` (5 ชุด) และ Transformation 4 แบบ (รวม PCA+L2 ที่ grid ไม่ได้รัน) ⟶ `transform_vectors` คำนวณสดทุกครั้ง
- ใช้ `rank_by_vector` บน `ratings` เต็ม (สาธิต ไม่ใช่ประเมิน) ; α, β ใช้ค่าเริ่มต้น 1 (ไม่มี slider ในเล่มนี้)
- แสดง History, Profile 20 มิติแรก และ Top-K พร้อม Score ; ไม่มีสื่อของอันดับ 1 (ต่างจาก LAB 1 / LAB 3)
- แนวทางสอน: ผู้ใช้เดิม ⟶ สลับ Text-SBERT ⟷ Hybrid ⟶ ดูว่า Top-K เปลี่ยนไหม ⟶ สลับ L2 ⟷ Center+L2 ⟶ เชื่อมกับตาราง `view` ของ Part 8

Part 10: gui_exp — Multimodal Experiment Explorer (2/2)

```
            p[:20]}), pd.DataFrame(rec)

demo = gr.Interface(
    gui_exp,
    [gr.Dropdown(sorted(ratings["user_id"].unique()), label="User"),
     gr.Dropdown(list(CONFIGS), value="Hybrid-Multimodal",
             label="Vector Configuration"),
     gr.Dropdown(["L2", "Center+L2", "Standard+L2", "PCA+L2"],
             value="L2", label="Transformation"),
     gr.Dropdown(PROFILE_METHODS, value="Rating-centered",
             label="Profile"),
     gr.Slider(1, 5, 3, 1, label="Top-K")],
    [gr.Dataframe(label="History"), gr.Dataframe(label="Profile"),
            gr.Dataframe(label="Recommendations")],
    title="Multimodal Experiment Explorer")
demo.launch(debug=False)
```

อธิบายโค้ด

สำรวจ configuration ด้วยตา

- เลือก Config จาก `CONFIGS` (5 ชุด) และ Transformation 4 แบบ (รวม PCA+L2 ที่ grid ไม่ได้รัน) ⟶ `transform_vectors` คำนวณสดทุกครั้ง
- ใช้ `rank_by_vector` บน `ratings` เต็ม (สาธิต ไม่ใช่ประเมิน) ; α, β ใช้ค่าเริ่มต้น 1 (ไม่มี slider ในเล่มนี้)
- แสดง History, Profile 20 มิติแรก และ Top-K พร้อม Score ; ไม่มีสื่อของอันดับ 1 (ต่างจาก LAB 1 / LAB 3)
- แนวทางสอน: ผู้ใช้เดิม ⟶ สลับ Text-SBERT ⟷ Hybrid ⟶ ดูว่า Top-K เปลี่ยนไหม ⟶ สลับ L2 ⟷ Center+L2 ⟶ เชื่อมกับตาราง `view` ของ Part 8

สรุปแนวคิดของ Source Code Multimodal LAB 2

Take-away

- **ไม่เพิ่ม engine เพิ่มเงื่อนไข:** Core / Encoders / Fusion เดิม + dict ใหม่ `SPLITS`, `CONFIGS` และ `RUN_*` ⟶ experiment matrix 5 มิติ 120 แถว
- **ออกแบบ config ให้ตอบคำถามเป็นครอบครัว:** Text-only (TF-IDF, SBERT) vs CNN-family vs Transformer-family vs Hybrid — ทุกชุด `early_fusion` เท่ากันแล้ว L2
- **`evaluate_vector_space` ครั้งเดียว 120 ครั้ง:** k, n_neg, seed คงที่ ; `label` ระบุที่มาของแถว
- **Fair view ก่อนสรุป:** กรอง LOO + Full Catalog แล้วเรียง nDCG@3 ; อ่านคู่ Coverage/Gini/ILD/Novelty/Calibration และรู้ว่า ILD ข้าม space เทียบไม่ตรง
- **Early vs Late บน encoder ชุดเดียวกัน** ทำให้ผลต่างมาจาก fusion strategy เท่านั้น
- **ตีความอย่างเป็นธรรม:** TextCNN อ่อนทำให้ CNN-Multimodal เสียเปรียบ ; Sampled Negatives ≈ Full เมื่อ catalog เล็ก ; 12 เรื่องผลแกว่ง — ใช้เรียนวิธีคิด ไม่ใช่สรุปว่า encoder ใดดีที่สุด

LAB 2 = controlled experiment สำหรับ multimodal content-based recommender